

EDS Lab Assignments

Practical1

Name:Vedant Deshmukh

PRN : 202501040046

Calculate Momentum

Problem Statement:

Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum is calculated using the formula: $p = m * v$ where:

m is the mass of the object (in kilograms).
 v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Code:

```
mass = float(input())
velocity = float(input())
momentum = mass * velocity
print(f"{momentum:.2f} kgm/s")
```

Output:

The screenshot displays the CODETANTRA IDE interface. On the left, the problem statement for '1.1. Calculate Momentum' is visible, including the formula $p = m \times v$ and the input/output formats. The main editor shows the Python code for calculating momentum. The right sidebar displays the test results, indicating that 2 out of 2 shown test cases passed and 2 out of 2 hidden test cases passed. The test cases show expected and actual outputs for two different inputs.

Test Case	Expected Output	Actual Output
Test case 1	58.00	58.00
Test case 2	58.00 kg/s	58.00 kg/s

2. Conditional Calculation based on the Number of Digits

Problem Statement:

Write a Python program that accepts an integer as input. Depending on the number of digits in.

Constraints: $1 \leq n \leq 999$

Input Format:

- The input consists of a single integer.

Output Format:

- If is a single-digit number, print its square.
- If is a two-digit number, print its square root (rounded to two decimal places).
- If is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid"

Code:

```
import math
n=int(input())
sq = n*n
sq_root = math.sqrt(n)
cb_root = math.cbrt(n)
if n>1 and n<=9:
    print(f"{sq}")
elif n>=10 and n<=99:
    print(f"{sq_root:.2f}")
elif n>=100 and n<=999:
    print(f"{cb_root:.2f}")
else:
    print("Invalid")
```

Output:

The screenshot displays a coding environment with the following components:

- Header:** CODETANTRA logo, navigation links (Home, Learn Anywhere), user information (202501040046@mitaoe.ac.in), and links for Support and Logout.
- Problem Statement:** "1.1.2. Conditional Calculation Based on the Number of Digits". It includes the same constraints and input/output formats as the text above.
- Code Editor:** Contains the Python code provided in the previous blocks, with line numbers 1 through 12.
- Test Results:** A summary bar shows "4 out of 4 shown test case(s) passed" and "3 out of 3 hidden test case(s) passed". Below this, individual test cases are listed with their status (passed) and a comparison between expected and actual output (both are 81 for Test Case 1).
- Footer:** Navigation buttons for Previous, Next, and a Submit button.

3. Days Between Two Dates

Problem Statement:

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Code:

```
from datetime import date
from datetime import datetime
date1 = input().strip()
date2 = input().strip()
d1 = datetime.strptime(date1, "%Y-%m-%d")
d2 = datetime.strptime(date2, "%Y-%m-%d")
difference = d2-d1
print(difference.days)
```

Output:

The screenshot shows a coding platform interface with a problem statement, input/output formats, and a code editor. The code is a Python program that calculates the number of days between two dates. The program is tested and passes 2 out of 2 test cases.

Problem Statement: Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format YYYY-MM-DD.
- The second line contains the second date in the format YYYY-MM-DD.

Output Format:

- An integer representing the number of days between the given dates.

Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

Code:

```
1 from datetime import date
2 from datetime import datetime
3 #write your code here...
4 date1 = input().strip()
5 date2 = input().strip()
6 d1=datetime.strptime(date1, "%Y-%m-%d")
7 d2=datetime.strptime(date2, "%Y-%m-%d")
8 difference=d2-d1
9 print(difference.days)
```

Test Results:

- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

Test Case 1:

Expected output	Actual output
2014-07-20	2014-07-20
2014-11-20	2014-11-20
123	123

Test Case 2:

Expected output	Actual output
2014-07-20	2014-07-20
2014-11-20	2014-11-20
123	123

4. Reverse a Number

Problem Statement:

Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Code:

```
n=int(input())
reverse=int(str(n)[::-1])
print(reverse)
```

Output:

The screenshot displays a coding environment with the following components:

- Header:** CODETANTRA logo, navigation links (Home, Learn Anywhere), user profile (202501040046@mitaoe.ac.in), and a Logout button.
- Problem Statement (1.1.4. Reverse a Number):** "Write a Python program to reverse the digits of the number and print the reversed number." It includes input and output format instructions and a section for sample test cases.
- Code Editor:** A Python script titled "reverseN..." with the following code:

```
1 #write your code here...
2 n=int(input())
3 if n<0:
4     reverse=int(str(abs(n))[::-1])
5 else:
6     reverse=int(str(n)[::-1])
7 print(reverse)
```
- Test Results:** A summary showing "2 out of 2 shown test case(s) passed" and "3 out of 3 hidden test case(s) passed". It includes a table for Test case 1 with expected and actual outputs.
- Footer:** A terminal window and navigation buttons (Prev, Reset, Submit, Next).

Test case	Expected output	Actual output
Test case 1	5347	5347
	7635	7635

5. Multiplication Table

Problem Statement:

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:
<number> x <multiplier> = <result>

Code:

```
n = int(input())  
  
for i in range(1,11):  
    print(n, "x", i, "=", n*i)
```

Output:

The screenshot displays a coding interface with the following components:

- Problem Statement:** Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.
- Input Format:** The first line of input contains an integer that represents the number for which the multiplication table is to be printed.
- Output Format:** Print the multiplication table for the given number in the format: **<number> x <multiplier> = <result>**
- Note:** The character "x" is used in the output to represent multiplication. Refer to the visible test-cases for more clarity.
- Sample Test Cases:** A section for test cases, currently empty.
- Code Editor:** Contains the following Python code:

```
1 #write your code here...  
2 n=int(input())  
3  
4 for i in range(1, 11):  
5     print(f"{n} x {i} = {n*i}")  
6  
7  
8  
9  
10
```
- Test Results:** Shows that 2 out of 2 shown test case(s) passed and 2 out of 2 hidden test case(s) passed. The average time is 0.003 s and the maximum time is 0.003 s.
- Test Case 1:** Displays the expected output and actual output for the input 8.

Expected output	Actual output
8 x 1 = 8	8 x 1 = 8
8 x 2 = 16	8 x 2 = 16
8 x 3 = 24	8 x 3 = 24
8 x 4 = 32	8 x 4 = 32
8 x 5 = 40	8 x 5 = 40

1.2.1 Pass or Fail

Problem Statement:.

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer, the number of courses.
- The second input will be integers representing the marks of the student in each of the courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Code:

```
def calculate(marks, total_courses):
    if any(mark<40 for mark in marks):
        return "Fail"
    total_marks = sum(marks)
    aggregate_percentage = ((total_marks)/(total_courses*100))*100
    if aggregate_percentage > 75:
        grade = "Distinction"
    elif aggregate_percentage >= 60:
        grade = "First Division"
    elif aggregate_percentage >= 50:
        grade = "Second Division"
    elif aggregate_percentage >= 40:
        grade = "Third Division"
    return (aggregate_percentage, grade)
num_courses = int(input())
marks = list(map(int,input().split()))
result = calculate(marks, num_courses)
if result == 'Fail':
    print("Fail")
else:
    aggregate_percentage, grade = result
    print("Aggregate Percentage:",f"{aggregate_percentage:.2f}")
    print(f"Grade: {grade}")
```

Output:

1.2.1. Pass or Fail

43/15

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing by the number of courses.

Sample Test Cases

passorFa...

Submit

```
1 # write your code here
2 n=int(input())
3 marks=list(map(int,input().split()))
4 failed = False
5 for m in marks:
6     if m < 40:
7         failed = True
8     break
9 if failed :
10     print("Fail")
11 else :
12     total=sum(marks)
13     agg=total/n
14
15     print(f"Aggregate Percentage: {agg:.2f}")
16 if agg > 75:
17     print("Grade: Distinction")
18 elif (agg>=60):
19     print("Grade: First Division")
20
```

Average time

0.003 s

3.30 ms

Maximum time

0.005 s

5.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1

Debug

Expected output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Actual output

4

55 65 78 89

Aggregate Percentage: 71.75

Grade: First Division

Test case 2

Terminal

Test cases

< Prev Reset Submit Next >

1.2.2 Fibonacci Series

Problem Statement:

Write a Python program that uses recursion to print the first terms of the Fibonacci series.

Input Format:

- A single integer representing the number of terms to generate.

Output Format:

- A single line containing the first terms of the Fibonacci sequence, separated by spaces.

Code:

```
def fibonacci(n, a=0, b=1):  
    if n==0:  
        return a  
    else:  
        return fibonacci(n-1,b,a+b)
```

```
n=int(input())  
for i in range(n):  
    print(fibonacci(i),end=" ")
```

Output:

The screenshot displays the CodeTANTRA IDE interface. On the left, the problem statement for '1.2.2. Fibonacci Series' is visible, including the input and output formats. The main editor shows the following Python code:

```
1 def fibonacci(n):  
2     if n==1:  
3         return 0  
4     # write the code..  
5     elif n==2:  
6         return 1  
7     else:  
8         return fibonacci(n-1)+fibonacci(n-2)  
9  
10  
11 n = int(input())  
12 for i in range(1, n+1):  
13     print(fibonacci(i), end=" ")  
14  
15
```

Below the code editor, the test results are shown. The average time is 0.008 s and the maximum time is 0.020 s. Two test cases are passed, with the expected output being '0 1 1 2 3' and the actual output being '0 1 1 2 3'.

1.2.3 Pattern-1

Problem Statement:

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Code:

```
rows = int(input())
for i in range(1, rows+1):
    for j in range(i):
        print("*",end = "")
    print()
```

Output:

The screenshot shows the CODETANTRA online code editor interface. The left sidebar contains the problem description for "1.2.3. Pattern - 1". The main editor area displays a Python program that takes an integer input and prints a right-angled triangle pattern of asterisks. The bottom right panel shows the execution results, including average and maximum execution times, and a summary of test cases passed.

1.2.3. Pattern - 1

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

rightangl...

```
1 #Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     for j in range(1,i+1):
5         print("*",end=" ")
6     print()
```

Average time: 0.002 s
Maximum time: 0.003 s

2 out of 2 shown test case(s) passed
4 out of 4 hidden test case(s) passed

Test case 1

Expected output

```
*
* *
* * *
```

Actual output

```
*
* *
* * *
```

Terminal Test cases

< Prev Reset Submit Next >

1.2.4 Pattern-2

Problem Statement:

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

Code:

```
n=int(input())
for i in range(1, n+1):
    for j in range(1, i+1):
        print(j,end=" ")
    print()
```

Output:

The screenshot shows the CodeTANTRA online IDE interface. On the left, the problem description for '1.2.4. Pattern - 2' is displayed, including the input format (an integer representing the number of rows), the output format (a right-angled triangle pattern of numbers), and a note to refer to test cases. The main editor shows the following Python code:

```
1 # Type Content here...
2 n=int(input())
3 for i in range(1,n+1):
4     for j in range(1,i+1):
5         print(j,end=" ")
6     print()
7
```

Below the code editor, the execution results are shown. The average time is 0.002 s (2.00 ms) and the maximum time is 0.003 s (3.00 ms). The status indicates '2 out of 2 shown test case(s) passed' and '2 out of 2 hidden test case(s) passed'. A table shows the expected and actual output for Test case 1:

Expected output	Actual output
1 ->	1 ->
1 2 ->	1 2 ->
1 2 3 ->	1 2 3 ->
1 2 3 4 ->	1 2 3 4 ->
1 2 3 4 5 ->	1 2 3 4 5 ->

At the bottom, there are buttons for 'Terminal', 'Test cases', 'Prev', 'Reset', 'Submit', and 'Next'.